SnapMail

Master Build Prompt for an AI app builder

Disposable-email web app | Version 1.2 | 6 November 2026

Developed & Designed by **Malek Hassan Ashour**

0. How to use this document

- Paste the whole document into the builder (for example Google AI Studio Build Mode or Claude Code) in one message. If the message limit is too small, paste in section order and say that more follows.
- This document contains **instructions only**. The only code in it is the four mail.tm API calls in section 4. The builder writes all application code itself.
- The builder must work in the order of section 6, stop at every **Acceptance** line, and verify it before moving on.
- **Phase 1** is the whole product: inbox, countdown, message view, QR, Arabic and English. It uses no Firebase and shows no ads.
- **Phase 2** (optional Firebase sync) starts only after Phase 1 passes and the owner says go.
- Nothing here has been run against the live mail.tm service. The first real inbox created on a phone is the first real test.

1. Role and goal

You are a senior front-end engineer. Build **SnapMail**: a static single-page web application that gives a visitor a disposable email address in about two seconds, shows incoming messages live, and deletes the inbox when its countdown ends.

- Stack: React, TypeScript (strict mode), Vite, Tailwind CSS. No server, no database, no proxy.
- Mail engine: the public mail.tm REST API, called directly from the browser (section 4).
- Audience: everyday users who want to avoid spam, developers and testers who need repeated throwaway inboxes, and Arabic-speaking mobile users.
- Language: Arabic (right to left) and English. Default from the browser language.
- Look: clean and light, using only white and blue; the name SnapMail in bold italic English (section 8).

2. Non-negotiable rules

1. **No backend.** No server code, serverless function or proxy. The browser calls https://api.mail.tm directly.
2. **There is no API key.** Never ask the owner for one and never invent one. The only credential is the per-inbox token returned by the token call.
3. **Domains are dynamic.** Always take the domain from the domains call. Never hard-code one.
4. **Secrets stay in the browser.** The mailbox password and token live only in session storage (or local storage if the user turns on Keep this inbox). Never put them in Firebase, a URL, a log line or a QR code.
5. **Ads are off.** Add an ad-slot component that renders nothing unless the environment variable VITE_ADS_ENABLED is the string 'true'. The default is false. Do not add any ad-network script, pop-under, push-notification or interstitial code.
6. **No payment code.** No payout, wallet, crypto or checkout code anywhere in the repository.
7. **Treat email HTML as hostile.** Sanitize with DOMPurify and render it in a sandboxed iframe with no allow-scripts and no allow-same-origin. Block remote images by default.
8. **Firebase is Phase 2 and optional.** When added, import it dynamically so visitors who never opt in never download it. The default flow makes no Firebase call.

9. **Direction.** Use Tailwind logical utilities (ms-, me-, ps-, pe-, text-start). The email address and verification codes always have dir=ltr, even in Arabic.
10. **Attribution.** The footer shows: Developed & Designed by Malek Hassan Ashour. Also set it as the package.json author and in the HTML meta author tag.
11. **Be honest about testing.** Never say something works unless you ran it. If your environment cannot reach api.mail.tm, say so and finish with the manual test list from section 10.

3. Product behaviour in one paragraph

On first load the app creates an inbox automatically and shows the address, a Copy button, a QR button, a New inbox button and a countdown (default 10 minutes, Extend adds 10). Every 5 seconds while the tab is visible (every 30 seconds when hidden) it fetches the message list. Selecting a message shows it safely. When the countdown reaches zero the app deletes the mail.tm account, forgets the token and shows an Expired panel with a New inbox button. A page refresh keeps the inbox; closing the tab ends it.

4. The mail engine: mail.tm API

These four calls are the foundation of the whole site and the only code in this document. The base URL is `https://api.mail.tm`. Steps 1 to 3 need no authentication; step 4 needs the Bearer token from step 3. The address in step 2 must use a domain returned by step 1.

```
# 1. Get available domains
curl https://api.mail.tm/domains

# 2. Create account
curl -X POST https://api.mail.tm/accounts \
-H "Content-Type: application/json" \
-d '{"address":"user@domain.com","password":"secret"}'

# 3. Get token
curl -X POST https://api.mail.tm/token \
-H "Content-Type: application/json" \
-d '{"address":"user@domain.com","password":"secret"}'

# 4. Fetch messages
curl https://api.mail.tm/messages \
-H "Authorization: Bearer TOKEN"
```

Use the four calls above exactly. The table below lists the other requests the app needs, which follow the same pattern.

Need	Request	Notes
List domains	GET /domains	Response is a Hydra collection: read `hydra:member`. Pick the first entry with isActive true and isPrivate false.
Create inbox	POST /accounts	Body: address and password. Generate both with the browser's secure random generator (crypto.getRandomValues), never Math.random.
Get token	POST /token	Returns id and token. Send as Authorization: Bearer TOKEN.
List messages	GET /messages?page=1	Needs token. First page only in Phase 1.
Read message	GET /messages/{id}	Needs token. Full text and html (html is an array of strings).
Mark seen	PATCH /messages/{id}	Body {seen:true}, Content-Type application/merge-patch+json.
Delete message	DELETE /messages/{id}	Needs token.
End inbox	DELETE /accounts/{id}	Needs token. Permanent.

- Rate limit: 8 requests per second per IP address. HTTP 429 means back off: wait 2, 4, 8 seconds (capped at 30) plus a small random delay, and give up after 4 attempts.
- HTTP 401 on a message call: request a new token once with the stored address and password. If that fails, treat the inbox as expired.
- Inspect real responses and use their actual field names. If a response differs from what this document says, follow the real response and tell the owner.

- Do not describe mail.tm as having an API key or a registration step. It has neither for these calls.

5. Project structure

Organize the source under src/ into these folders. File names are suggestions; the separation is the requirement.

- **types**: data shapes for the mail.tm responses (domain, account, token, message summary, message detail, attachment) and for the app (locale, ad placement, inbox session).
- **services**: one mail service module. Every mail call goes through one provider interface, so mail.tm can be replaced later by editing one file.
- **hooks**: an inbox hook and a countdown hook (section 6, steps 3 and 5).
- **lib**: retry-with-backoff helper, session storage helper, language dictionary, safe-HTML helper, verification-code finder.
- **components**: address bar, countdown ring, message list, message view, QR dialog, expired panel, error banner, language toggle, ad slot, footer.
- **styles**: one CSS file with the Tailwind layers and the card style (section 8).
- Root files: index.html (author meta tag, language and direction set at runtime), Vite config, Tailwind config, TypeScript config (strict mode and noUncheckedIndexedAccess on), and a hosting config for single-page rewrites and security headers (step 10).

6. Build steps (do them in this order)

Step 1. Scaffold

- Create a Vite React TypeScript project. Install: tailwindcss, dompurify, qrcode (with its types), and the @fontsource packages for IBM Plex Sans Arabic, JetBrains Mono and Inter.
- Turn on TypeScript strict mode and noUncheckedIndexedAccess. Put the colors, fonts, radius, shadows and transition times from section 8 into the Tailwind theme.
- Self-host fonts through @fontsource. Do not load fonts from a third-party CDN.

Acceptance: The production build succeeds with no TypeScript errors, and the page shows a white background with the SnapMail name in blue italic.

Step 2. Types, mail service, helpers

- Define the data shapes. A mail message summary has: id, from (address and name), to, subject, intro, seen, hasAttachments, size, createdAt. A message detail adds: text, html (array of strings) and attachments (id, filename, contentType, size). Verify the exact field names against real responses.
- An inbox session holds: accountId, address, password, token, createdAt and expiresAt (both as epoch milliseconds). It lives only in the browser.
- Write one mail service with these operations: create inbox, refresh token, list messages, get message, mark seen, delete message, delete inbox. It exposes them through a single provider interface.
- Create inbox does the section 4 sequence: list domains, choose the first active public one, generate a random 10-character address and a random 24-character password, create the account, request the token, and set expiresAt from the chosen countdown length.
- Use one shared request function that adds the JSON headers and the Bearer token, and turns any non-success response into an error carrying the HTTP status.
- Write a retry helper that retries only on HTTP 429, with the backoff from section 4.
- Write a storage helper that loads, saves and clears the session. It reads session storage first, then local storage. Every storage call is wrapped so that blocked storage never crashes the app (the inbox then lives in memory only).

Acceptance: The type checker passes. If the environment can reach api.mail.tm, run a script that creates an inbox, lists messages and deletes the account, and report the result.

Step 3. Inbox hook

- Expose: the session, the message list, a status (idle, creating, ready, expired, error), an error text, and the actions create, end and extend.
- Create: sets status to creating, calls the service through the retry helper, stores the session, sets status to ready. On failure set status to error with a short message that includes the HTTP status when there is one.
- Poll loop while ready: check expiry first (if past expiresAt, end the inbox as expired); otherwise list messages. Schedule the next poll for 5 seconds if the tab is visible and 30 seconds if hidden. Cancel cleanly on unmount or inbox change.
- On HTTP 401, request a new token once and continue; if that also fails, end the inbox as expired. On other errors show the connection-problem text and keep polling.
- End: clears storage and state first, then tries to delete the mail.tm account. If deletion fails, ignore it (the provider cleans abandoned accounts).
- Extend: sets expiresAt to now plus 10 minutes and saves the session.
- In App, call create once on mount when the status is idle.

Acceptance: On load an address appears; closing and reopening the tab starts a new inbox; refreshing keeps the same one; the app sends no requests after expiry.

Step 4. Countdown hook

- Compute remaining time from expiresAt minus the current time on every tick. Never subtract one second per tick, because browsers slow timers in background tabs. Also recalculate when the tab becomes visible again.
- Return remaining milliseconds, an mm:ss label and a level: ok, warn (2 minutes or less), critical (30 seconds or less), expired (zero).

Acceptance: After switching to another tab for a minute and coming back, the countdown shows the correct time.

Step 5. Language and direction

- Build the Arabic and English dictionaries and a language hook. Set the document language and direction on change. Remember the choice in local storage. Default from the browser language.
- Required keys: appName, yourAddress, copy, copied, newInbox, extend, expiresIn, waiting, noSubject, from, loadImages, imagesBlocked, qrTitle, close, expired, createFailed, connectionProblem, advertisement, footerCredit.
- All user-visible strings come from the dictionary. No hard-coded copy in components.

Acceptance: Switching to Arabic mirrors the whole layout; the email address stays left to right.

Step 6. Address bar and countdown ring

- Address bar: address in 20 px monospace (24 px on desktop), buttons Copy, QR, New inbox, each at least 44 px tall. Copy uses the browser clipboard and shows Copied for 1.5 seconds.
- Countdown ring: SVG ring plus the mm:ss text. Colors by level follow section 8: ok = blue-600, warn = blue-700 with the text Expiring soon, critical = error-600 with a pulse (no pulse when reduced motion is on).
- Announce the warn level once through an aria-live polite region. Never announce every second.
- Extend adds 10 minutes through the inbox hook.

Acceptance: The ring changes color at 2:00 and 0:30; Extend adds 10 minutes; a screen reader hears one announcement.

Step 7. Inbox list and message pane

- Below 768 px: single column. Tapping a message opens it full screen; use the browser history so the back button returns to the list.
- 768 px and up: split inbox, list 340 px wide, detail fills the rest.
- List item: sender, subject (or the noSubject text), intro line, relative time, unread dot. Empty state: the waiting text with a quiet animation (off under reduced motion). Loading skeleton while the first fetch runs.
- Error banner shows the hook's error text with a retry action; polling continues after errors.

- Verification codes: find 4 to 8 digit numbers in the plain-text body and show each as a one-tap Copy chip above the message.
- Opening a message reads it in full, then marks it seen, and updates the list locally.

Acceptance: A test email appears within about 5 seconds; opening it marks it seen; the code chip copies.

Step 8. Safe message rendering

- Sanitize with DOMPurify using the HTML profile, forbid style and form tags, and (when remote images are not allowed) remove every image source that is not a data: URL and set its alt text to the imagesBlocked text.
- Render the result in an iframe with sandbox set to allow-popups and allow-popups-to-escape-sandbox only, referrer policy no-referrer, and the content passed as srcdoc. Never add allow-scripts or allow-same-origin.
- A Load images button re-renders with remote images allowed for that message only. If the message has no HTML, show the plain text in a wrapped block.
- Attachments: list file names and sizes only. Do not link their download addresses directly.

Acceptance: A message containing a script tag and a remote image shows neither the script's effect nor the image until Load images is pressed.

Step 9. QR dialog

- Generate the QR code in the browser with the qrcode package (240 px, margin 1, error correction M) and show it as an image. Never call an external QR service.
- The code encodes the address only, never the token or password. Do not add a continue-on-another-device feature.
- Accessibility: dialog role, modal, focus moves into the dialog and returns on close, Escape closes.

Acceptance: Scanning the QR code with a phone camera yields exactly the address text.

Step 10. Expiry, ad stub and security headers

- At zero: end the inbox as expired (deletes the mail.tm account, clears storage, stops polling) and show the Expired panel with New inbox.
- New inbox while one is active: end it first, then create.
- Keep this inbox switch: moves the session to local storage; turning it off moves it back to session storage.
- Ad slot component: renders nothing unless VITE_ADS_ENABLED is the string 'true'. Even when true, in Phase 1 it renders only an empty labelled placeholder. Real ads are Phase 3, only after the owner confirms in writing that ads are allowed, and they must load in a sandboxed iframe from a separate static page, never in the application's own origin.
- Hosting config: rewrite every path to index.html. Send these headers on all pages: a Content-Security-Policy that allows only the site itself, plus api.mail.tm for connections, data: and https: for images, no inline scripts, no framing and no base tag; Referrer-Policy no-referrer; X-Content-Type-Options nosniff; Permissions-Policy that disables camera, microphone and geolocation. Keep the policy as one line in the real file.

Acceptance: After expiry no further requests are sent and the old token no longer reads mail. The default build contains no ad-network domain and no third-party script. The browser console shows no Content-Security-Policy violations.

Step 11. Accessibility and polish

- Touch targets at least 44 px. Visible focus ring (blue-600, 2 px offset). Logical tab order.
- Respect prefers-reduced-motion. Check text contrast for every text and background pair against WCAG AA (4.5:1 for body text) and report any failure.
- Set the page title and a meta description in both languages. Add a simple SVG favicon.

Step 12. Verify and report

- Run the checklist in section 10. Then write the report described in section 11.

7. Phase 2 (only on the owner's go-ahead): optional Firebase sync

Purpose: remember language and default countdown length, and keep a short history of addresses. Never store passwords, tokens, message data or sender addresses.

- Use the Spark (free) plan. Anonymous sign-in happens only when the user turns on sync, because new-account creation is limited per IP address and mobile carriers share addresses.
- Import Firebase modules dynamically. Configuration comes from four environment variables (API key, auth domain, project id, app id). They are public configuration, not secrets.
- Never read Firestore to show mail. Never poll Firestore. Keep at most 20 history documents per user.
- **Data model:** one settings document per user (locale ar or en, default countdown minutes from 1 to 60, created time) and, under it, one document per inbox named by the mail.tm account id (address, created time, expiry time, status active, expired or deleted).
- **Security rules:** a signed-in user can read, write and delete only their own documents. Writes may contain only the fields listed above, with the value limits listed above and an address no longer than 254 characters. Everything else is denied.

Acceptance: A write with an extra field is rejected; an unauthenticated read is rejected; a user cannot read another user's documents.

8. Design system: white and blue

The whole interface uses only white and blue. The single exception is a muted red used for errors and the last 30 seconds of the countdown, so that danger is not shown by a shade of blue alone.

Token	Value	Used for
white	#FFFFFF	Page background, cards, message body
sky-50	#F0F7FF	Page sections, input fields, empty states
sky-100	#DBEAFE	Hover, selected message, chips
sky-200	#BFDBFE	Borders, dividers, ring track
blue-600	#2563EB	Primary buttons, links, countdown ring, focus ring, brand name
blue-700	#1D4ED8	Pressed and hover state of blue-600, warn level
navy-900	#0F1B3D	Primary text
slate-500	#5B6B8C	Metadata and secondary text
error-600	#DC2626	Errors and countdown critical level only

- **Brand name:** The name SnapMail is always written in English, in **bold italic**, in blue-600, in the top-left of the header (top-right when the page is in Arabic). Use Inter Bold Italic through @fontsource/inter. Italic applies to the English brand name only, never to Arabic text. No other logo is needed.
- **Cards:** white fill, 1 px sky-200 border, radius 20 px, soft shadow (0 8px 24px blue-600 at 10% opacity). No background blur and no frosted glass. Controls use radius 12 px.
- **Buttons:** primary = blue-600 fill with white text; secondary = white fill, 1 px blue-600 border and blue-600 text; hover moves to blue-700 or sky-100. Every button is at least 44 px tall.
- **Countdown ring:** track in sky-200; ok level in blue-600; warn level (2 minutes or less) in blue-700 with the text Expiring soon; critical level (30 seconds or less) in error-600 with a pulse (no pulse under reduced motion).
- **Fonts:** IBM Plex Sans Arabic for text, JetBrains Mono for the address and codes, Inter Bold Italic for the brand name only. Body 15 px, line height 1.7 for Arabic and 1.5 for Latin. Metadata 12.5 px in slate-500.
- **Contrast:** Check every text and background pair against WCAG AA (4.5:1 for body text), especially slate-500 on sky-50, blue-600 on sky-100, and white on blue-600. Darken the color, not the font size, if a pair fails, and report it.
- **Motion:** 150 ms for small changes, 250 ms for panels. Everything animated must stop under prefers-reduced-motion.

- **Theme:** One light theme only. Do not build a dark mode or a theme switch in Phase 1.

9. Screens

Viewport	Layout
Below 768 px	Single column: address bar, countdown, list. A tapped message becomes a full-screen view with a back button. Footer last.
768 to 1279 px	Address bar and countdown across the top. Below: list 340 px on the start side, message pane on the rest.
1280 px and up	Same split, centered, 1200 px maximum. (An ad column may be added in Phase 3; reserve nothing for it now.)

- States to design and build: creating (skeleton), ready and empty, ready with messages, message open, error banner, expired, QR dialog open.
- Error copy must be actionable and available in both languages (for example: could not create an inbox, try again).

10. Definition of done

[ ] The production build and the type check pass with zero errors.
[ ] An inbox is created on load, with an address from a domain returned by GET /domains.
[ ] A real email sent to the address appears in the list within about 5 seconds and opens correctly.
[ ] The countdown, Extend, expiry and account deletion all work; no requests after expiry.
[ ] Refresh keeps the inbox; closing the tab ends it.
[ ] Works on a phone using mobile data, not only on Wi-Fi or a laptop.
[ ] Arabic layout mirrors correctly; address and codes stay left to right; English works too.
[ ] Script tags and remote images in a message are neutralized (Step 8 test).
[ ] The QR code decodes to the address only.
[ ] No ad-network domain, payment code, API key or Firebase call exists in the default build.
[ ] Bundle for the first load is under 150 kB gzipped, or the report states the real size.
[ ] Footer shows Developed & Designed by Malek Hassan Ashour.

Manual test list for the owner (use if your environment cannot reach mail.tm): open the deployed site on a phone with Wi-Fi off; copy the address; send an email to it from another account; confirm it arrives; open it; let the countdown reach zero; confirm the Expired panel appears.

11. Final report format

When finished, reply with exactly these parts: (1) the list of files created; (2) the commands you ran and their results; (3) the checklist from section 10 with pass, fail or not verified for each line; (4) anything you changed from this prompt and why; (5) anything you could not verify, including whether you reached api.mail.tm. Do not report a line as passed unless you ran it.

12. Out of scope (do not build)

- Any server, proxy or serverless function. Sending email. User accounts or passwords for SnapMail itself.
- Advertising code, payout or payment code, analytics that sends addresses or message data anywhere.
- A continue-on-another-device feature that carries the token.
- Anything that stores message content outside the browser.

13. Risks the owner has already been told about

Do not hide or remove these in the interface or the repository notes. Add them to a README section called Known risks.

- mail.tm is a free third-party service with no service-level agreement, and its usage rules, as quoted by a community library, forbid earning money from software that uses only its API. Ads stay off until the owner has written permission.
- Sites may block the mail.tm domains, which would make addresses unusable for sign-ups. Measure acceptance on a fixed list of services before promoting the product.
- If the site is later monetized, the free Vercel plan does not allow commercial use; a paid plan or another host is needed.